

Speaker script

Communications Systems Visualizer — ENGR 78 final presentation

Target: ~16 minutes Deck: `presentation.tex` (15 slides)

Howard Wang

Kaw Moo

Charlie Schuetz

Spring 2026

Speakers. Divide blocks as you prefer; timings assume one primary narrator unless you use the three-way split at the end.

Pacing. Speak each block as full sentences. If you run long, shorten screenshot slides by pointing at only two regions each.

Timing overview

#	Title	Target
1	Title	0:40
2	Motivation	1:30
3	Project objective	1:15
4	Overview: how the app works	1:15
5	Dashboard (QPSK)	1:35
6	Composite plot export	1:15
7	Full-dashboard capture (BPSK)	1:10
8	Constellation and eye (QPSK)	1:45
9	Tabs I–III	1:45
10	Tabs IV–V	1:45
11	Optional hardware path	1:15
12	Key files	0:55
13	Outcomes and extensions	1:00
14	Credits and codebase size	0:45
15	Thank you	0:20
Approximate total		~16:25

1 Slide 1 — Title

Target. ~40 seconds.

Good morning or good afternoon. We are Howard Wang, Kaw Moo, and Charlie Schuetz. Today we are presenting our ENGR 78 final project, the Communications Systems Visualizer.

It is an interactive web application built with Dash and Plotly. You run it locally in a browser; the goal is to tie together the main topics from the course—PCM, line coding, pulse shaping and eye diagrams, carrier modulation, and basic capacity and coding—in one consistent interface.

We will start with why we built it, then walk through screenshots of the running app, summarize each tab, mention an optional Pluto hardware path, and close with outcomes. We are happy to take questions at the end.

Do. Make eye contact; gesture once toward the title on screen. Optional handoff: “I will hand the next section to [name] for motivation.”

2 Slide 2 — Motivation

Target. ~1 minute 30 seconds.

Motivation first. ENGR 78 covers a lot of ground: sampling and quantization, base-band line codes, how you shape pulses and what inter-symbol interference does, pass-band modulation, and finally information limits and simple coding ideas.

In a typical week, those topics show up as separate homework sets or static figures in notes. That is fine for algebra, but it is harder to build intuition for how one knob—say bandwidth, roll-off, noise, or symbol rate—changes both the time waveform and the spectrum and the error behavior at the same time.

So we wanted one place where a student or instructor could move a slider or type a bit pattern and immediately see linked plots update: time domain, frequency domain, constellation or line waveform, and summary numbers. That is the educational gap we are addressing.

We also wanted it reproducible in a dorm or classroom: a locally hosted Dash app means you are not depending on a specific lab bench just to show a clean eye diagram or a line code spectrum.

Do. Emphasize the words “separate,” “one place,” and “linked” when they appear on the slide.

3 Slide 3 — Project objective

Target. ~1 minute 15 seconds.

This slide clarifies the project objective and the scope of what we actually built.

First, the core deliverable is a browser-based learning visualizer for ENGR 78. It is not meant to be a Pluto-only radio lab, and it is not just a set of separate static plots. The goal is to give students one consistent place to explore the course concepts.

Second, the app connects adjustable parameters to multiple views. Users can change things like bits, bandwidth, roll-off, noise, symbol rate, and modulation order, then immediately compare waveforms, spectra, eye diagrams, constellations, throughput, and capacity-style results.

Third, we kept the main demo laptop-only. You run `python main.py`, open localhost on port 8050, and the core project works without SDR hardware. The ADALM-PLUTO dashboard is still available through `pluto_live_app.py`, but it is a separate optional extension rather than the center of the project.

Do. Emphasize “browser-based,” “linked views,” and “laptop-only.”

4 Slide 4 — Overview: how the app works

Target. ~1 minute 15 seconds.

This slide shows the app in a simpler way: what the user changes, what the Python code does, and what appears on screen.

Starting on the left, the user interacts with a normal browser page: sliders, menus, and text inputs. The app itself is written in Python, using Dash for the web interface. Most of that logic lives in `web_app.py`, which reads the user's selections and decides which plots need to update.

The middle box is the signal math. Python libraries like NumPy and SciPy generate the communication signals: pulse-code modulation, line coding, pulse shaping, carrier modulation, spectra, eye diagrams, constellations, and capacity curves.

On the right, Plotly redraws the interactive graphs in the browser. The bottom box shows that our report and deck screenshots come from the running app. The top box shows the optional Pluto extension, which is a separate Python dashboard on port 8051 rather than part of the required laptop-only path.

Do. Trace the four main boxes left to right. Keep the message simple: user changes a setting, Python recomputes the signal, Plotly redraws the plots.

5 Slide 5 — Dashboard (carrier tab: QPSK)

Target. ~1 minute 35 seconds.

This is the main dashboard with the carrier tab active and QPSK selected. Treat this as the “home screen” mental model.

Across the top, you see the tab strip: PCM, line coding, ISI and eye, M-ary and carrier, capacity and codes. That ordering is deliberate—it roughly follows how the course introduces material.

Under the tabs, there is usually a row of summary statistics: rates, bandwidth hints, SNR or capacity readouts depending on the tab. Those numbers update when sliders move, so students get both plots and scalar checks together.

The large panels are Plotly figures. On this capture you can see multiple views at once—for example spectrum- and constellation-style panels depending on scroll position. Plotly gives you hover and zoom, which is surprisingly useful in office hours when someone asks what happens right here on the shoulder of the spectrum.

The important narrative is: one window, many linked views, immediate feedback when parameters change.

Do. Point at (1) tabs, (2) stats row, (3) one plot panel.

6 Slide 6 — Composite plot export

Target. ~1 minute 15 seconds.

This slide is a composite export—a single PNG that stitches several representative panels side by side. We used this a lot when writing the report and the plot guide, because it answers the question “what does the tool produce?” in one image.

You can see the constellation view, a spectrum-style view, an eye-style view, and throughput or time-series style panels, depending on how the export was framed. The pedagogical point is the same as before: students see several representations of the same underlying random data and pulse shaping.

Operationally, this kind of montage is also what you send to a TA or embed in documentation when you do not want a full scrolling browser capture.

Do. Trace left-to-right across the montage once, naming each region.

7 Slide 7 — Full-dashboard capture (BPSK)

Target. ~1 minute 10 seconds.

Here is a full vertical capture with BPSK selected. We show this to make two points.

First, the layout does not change when you switch modulation order; only the content of the plots and the numeric readouts change. That consistency lowers cognitive load—you are not relearning a new UI every week.

Second, BPSK is the easiest case to narrate in lecture: one bit per symbol, constellation on a line, spectrum symmetric in a simple way. When you later switch to 16-QAM in the same tab, students can focus on more points in the constellation, tighter spacing, higher bits per symbol, rather than on where the buttons moved.

If you are live-demoing, this is a good slide to say you will now switch to QPSK or 16-QAM in the app and watch the constellation density change, if you have the laptop connected.

Do. Contrast verbally with the next time you show a dense constellation (slide 8 or live demo).

8 Slide 8 — Constellation and eye diagram (QPSK)

Target. ~1 minute 45 seconds.

This slide includes one view that we did not focus on heavily in class: the constellation diagram on the left. I still wanted to include it because it is one of the most common ways engineers visualize digital modulation.

A constellation diagram is basically a map of possible transmitted symbols. The horizontal axis is the in-phase component, often called I , and the vertical axis is the quadrature component, called Q . For QPSK, there are four main symbol locations, so each point can represent two bits. The receiver’s job is to decide which of those four locations the received sample is closest to.

This connects to ideas we did learn: different modulation choices change how many bits each symbol can carry, and noise makes decisions harder. In a clean case, the received points cluster tightly near the ideal locations. As noise or channel impairments increase, the points spread out, so the receiver has a higher chance of choosing the wrong symbol.

The eye diagram answers a different question: do I have a clear vertical opening at the decision time, and are traces aligned across symbol periods? Closing the eye vertically usually means noise or amplitude problems; closing horizontally often points to timing or ISI.

So even though constellation diagrams were outside the main lecture sequence, they are a useful extension: the eye diagram shows timing and pulse-shape quality, while the constellation shows how separable the symbol decisions are.

Do. Point to the four QPSK clusters first, then to the eye opening. If short on time, skip the final sentence but keep the “map of symbols” explanation.

9 Slide 9 — Tabs I–III: baseband and channel

Target. ~1 minute 45 seconds.

We will now walk the first three tabs in a bit more detail—the baseband side of the course.

PCM tab: you choose a message bandwidth, an oversampling factor relative to Nyquist, and bits per sample. The app reports implied sampling rate, quantization levels, and PCM bit rate, and shows a conceptual sampled-and-quantized waveform. That supports the early part of the course where students practice counting bits and seeing aliasing intuition, without needing a separate toy script.

Line coding tab: you type a binary string—up to 32 bits in the UI—and pick among NRZ and RZ variants for on-off and polar signaling, bipolar AMI, and Manchester. The app plots the voltage versus time waveform and a windowed FFT magnitude for a PSD-style view. That connects time-domain duty cycle to frequency-domain nulls and lobes in a way a static figure rarely does.

ISI and eye tab: you control raised-cosine roll-off β , an optional neighbor-symbol coupling to mimic ISI, Gaussian noise, and PAM order. The eye is built from overlaid segments after matched filtering. Instructors can say: watch the eye close as I increase ISI or noise, and the class sees it immediately.

Do. Speak slowly on “NRZ/RZ/AMI/Manchester” so it does not sound like one word.

10 Slide 10 — Tabs IV–V: passband and information

Target. ~1 minute 45 seconds.

The last two tabs cover passband material and information material.

M-ary and carrier tab: you set PAM spacing and symbol rate, then choose among ASK and on-off keying, BPSK, binary FSK, QPSK, and 16-QAM. The app shows passband time waveforms, spectra, and constellation or amplitude views as appropriate. There is also a throughput view with instantaneous and cumulative bits so students connect symbols per second to bits per second concretely.

Capacity and codes tab: we plot the Shannon AWGN capacity

$$C = B \log_2(1 + \text{SNR})$$

versus SNR for a chosen bandwidth and mark the operating point. Separately, you can enter source probabilities and read entropy in bits. There is a small repetition-code toy where a received three-bit word is compared to codewords using Hamming distance, reinforcing distance to codeword as a decoding picture without building a full simulator lab in week one.

Implementation reminder: almost all callback logic lives in `web_app.py`. `main.py` is only the entry point that starts the Dash server.

Do. Write the Shannon formula on a board if available, or gesture at it on the slide; do not rush the repetition-code sentence.

11 Slide 11 — Optional hardware path

Target. ~1 minute 15 seconds.

We want to be explicit about scope for grading and for demos.

The required experience is the browser app we have been showing. The optional path is `pluto_live_app.py`, a second Dash application that streams IQ samples through a `RealtimeEngine` class in `src/realtime_engine.py`. In the live UI you typically switch among BPSK, QPSK, and 16-QAM, and you see spectrum, constellation, eye, and SNR or throughput trends updating over time.

If you run this on macOS, the README notes that Pluto USB Ethernet should be in NCM mode and the device is often reachable at 192.168.2.1. We are not going to read that like a networking lecture—the point is: hardware has setup steps, so we isolated it behind a separate script so the main project still runs on any laptop.

Closing message for this slide: grading can rely entirely on the browser tool; Pluto is value-added for instructors who want a live RF story in the last few minutes of class.

Do. Nod to whoever handled Pluto integration if applicable.

12 Slide 12 — Key files

Target. ~55 seconds.

Here is the minimal file map if someone clones the repo tomorrow.

`main.py` is the entry point; it starts Dash on localhost, default port 8050.

`web_app.py` holds the tabs, the callbacks, and the NumPy and Plotly figure generation for the class visualizer.

`pluto_live_app.py` is the optional live Pluto dashboard.

`requirements.txt` pins NumPy, SciPy, Dash, Plotly, and pyadi-iio for Pluto support. That is the whole install story for the teaching path.

Do. Read the table row by row; do not improvise long filenames here.

13 Slide 13 — Outcomes and extensions

Target. ~1 minute.

Outcomes: we delivered a unified interactive visualizer whose tab order tracks the ENGR 78 topic progression. Students can explore relationships among rate, bandwidth, noise, and constellation geometry without installing a heavy SDR toolchain for the core experience.

Extensions we would consider next if we continued: saved presets per lecture so an instructor can jump to “week five eye diagram demo” instantly; CSV export of traces for lab reports; more line codes or decoders; and deeper forward error control labs that still reuse the same Dash shell.

Do. Keep this as the closing technical summary; the next slide handles credits.

14 Slide 14 — Credits and codebase size

Target. ~45 seconds.

We also want to include a short credits slide for transparency.

The project team is Howard Wang, Kaw Moo, and Charlie Schuetz. We also used AI assistance during the project: GPT 5.5 and Cursor Composer helped with code drafting, debugging, documentation, and presentation refinement.

For scale, the application source code is about 4,200 lines, excluding virtual environments and generated files. That count includes the main Python app, the `src/` modules, CSS, and source helper scripts.

Do. Say this cleanly and briefly; do not over-explain the line-count method unless asked.

15 Slide 15 — Thank you

Target. ~20 seconds.

Thank you for your attention. We will take questions now—and if it helps, we can also show the live app at the podium for two or three quick slider moves.

Do. Step forward; smile; invite questions clearly.

Optional three-way split (~5 minutes each)

1. **Speaker A (e.g. Howard)** — Slides 1–4: framing, motivation, objectives, architecture story.
2. **Speaker B (e.g. Kaw)** — Slides 5–8: all screenshots; guided tour of the UI.
3. **Speaker C (e.g. Charlie)** — Slides 9–15: tab detail, Pluto, files, outcomes, credits, close.

Rehearse handoffs on one sentence each, e.g. “I will let Kaw walk the screenshots.”

Backup lines if Q&A is quiet

- “We can show line coding changing the PSD in ten seconds if we plug in the laptop.”
- “The ISI slider is deliberately synthetic—it is for intuition, not for claiming a specific multipath channel model.”
- “Plotly runs in the browser, but the DSP runs in Python on the server; that is why very large FFTs can feel slower on old laptops.”

After the talk

- “Slides and report PDFs are under `docs/`; figures under `docs/plot_guide_screenshots/`.”
- “Run `make -C docs` from the repo root to rebuild the PDFs after edits.”